

Prelo

Manual técnico do projeto

Livraria online com catálogos, categorias, carrinho, checkout, pagamentos e rastreio de envio. Java 21, Spring Boot, JPA, Spring Security, Flyway, Redis e React, em PostgreSQL, MySQL, SQL Server ou Oracle.

Copyright (©) 2026 Clara Portes

Versão 1.0.0 · Outubro de 2026 · Licença MIT

Sumário

Visão geral 4
- Para que serve 4
- O que o Prelo faz 4
- Tecnologias 4
- Identidade visual 5

Como rodar 6
- Requisitos 6
- Primeira execução 6
- Escolher o banco 6
- Contas de demonstração 7
- Comandos do dia a dia 7
- Usar de verdade 8
- Rodar sem Docker 8
- Acesso pela rede e HTTPS 8

Guia das telas 9
- Vitrine 9
- Catálogo e busca 10
- Categorias e catálogos 11
- Página do livro 12
- Carrinho 13
- Entrar e criar conta 13
- Checkout 14
- Janela de pagamento 14
- Minhas compras 16
- Detalhes do pedido e do envio 17
- Minha conta 18
- Tema escuro e celular 19

Administração 20
- Painel 20
- Pedidos e envios 21
- Livros e estoque 22
- Categorias e catálogos 23

Pagamentos e envio 24
- Sandbox e cartões de teste 24

Como o cartão é protegido 24
Idempotência 24
Plugar um provedor real 24
Frete 25
Prazos e cancelamento automático 25
Cupons 25

Arquitetura 26
Pacotes da API 26
Segurança 26
Redis 27
Estrutura de pastas 27

API REST 28
Autenticação 28
Rotas principais 28
Exemplos 29
Erros 30

Configuração 31
Aplicação 31
Banco e Redis 31
Pagamentos 32
Checklist para produção 32

Testes e integração contínua 33
Testes da API 33
Testes da interface 33
Integração contínua 33

Solução de problemas 35

CAPÍTULO 1

Visão geral

O Prelo é uma livraria online completa: da vitrine ao rastreio do pacote. Você navega por catálogos e categorias, busca sem se preocupar com acentos, monta o carrinho, escolhe o frete, paga com cartão, Pix ou boleto e acompanha cada etapa do envio. A livraria tem um painel para cuidar de pedidos, estoque e vitrines.

Para que serve

O projeto foi pensado como uma base real de e-commerce em Java: pode ser usado como está para uma livraria pequena, estudado como referência de arquitetura (camadas, segurança, cache, concorrência de estoque, idempotência de pagamento) ou adaptado para outro tipo de loja. O mesmo código roda em **PostgreSQL, MySQL, SQL Server ou Oracle**, trocando uma variável.

O que o Prelo faz

- **Catálogos e categorias:** vitrines curadas com ordem própria (Mais vendidos, Lançamentos, Clássicos essenciais, Para programar melhor, Leituras de fim de semana) e oito categorias com contagem de livros.
- **Busca:** por título, subtítulo, autor, editora e ISBN, sem diferenciar acentos nem maiúsculas, com filtros de categoria, formato, preço e estoque, seis ordenações e paginação.
- **Página do livro:** capa gerada na paleta do projeto, sinopse, ficha técnica, estoque, cálculo de frete pelo CEP e livros da mesma estante.
- **Carrinho:** funciona sem login (fica no navegador) e é somado ao da conta quando você entra. Tem gaveta lateral, barra de frete grátis e cupons.
- **Checkout:** endereço com preenchimento pelo CEP, três formas de envio com preço e prazo por região, cupom e reserva do estoque por 60 minutos.
- **Janela de pagamento:** cartão de crédito em até 6x sem juros, Pix com QR Code e copia e cola, boleto com linha digitável. Modo sandbox para testar sem dinheiro de verdade.
- **Minhas compras e detalhes de envio:** filtros por status, barra de progresso, código de rastreio, previsão de entrega e linha do tempo do pacote.
- **Administração:** painel de vendas, pedidos e envios, livros e estoque, categorias e catálogos.
- **Interface fluida:** transições suaves, microinterações, tema claro e escuro e layout para celular.

Tecnologias

Camada	Tecnologia	Papel
API	Java 21 · Spring Boot 4.1 · Spring MVC	Rotas REST, validação, documentação OpenAPI (Swagger)
Segurança	Spring Security 7 · JWT (Nimbus, HS256) · BCrypt	Login, sessão sem estado, papéis CUSTOMER e ADMIN
Persistência	Spring Data JPA · Hibernate 7	Entidades, consultas, paginação, bloqueio otimista

Camada	Tecnologia	Papel
Migrações	Flyway 12	Uma pasta por banco em `db/migration/{banco}`
Bancos	PostgreSQL · MySQL · SQL Server · Oracle	Fonte da verdade: contas, catálogo, pedidos, pagamentos, envios
Cache e tokens	Redis 7 (Lettuce)	Cache do catálogo, sessões, tokens de cartão, limite de login, travas
Interface	React 19 · React Router 7 · Vite	Loja, checkout, compras, conta e administração
Entrega	nginx · Docker Compose	Serve a interface, repassa `/api` e sobe tudo com um comando
Testes	JUnit 5 · MockMvc · Vitest · GitHub Actions	Integração nos quatro bancos e componentes da interface

Identidade visual



A marca é a **Pilha**: três livros empilhados (preto, rosa e mostarda) com um ponto rosa no alto, como quem acabou de colocar mais um na pilha. O nome vem da expressão no prelo, o livro que está para sair. A interface usa a **Fraunces** nos títulos e a **Manrope** no texto.

Tinta	Névoa	Rosa	Mostarda	Pedra	Manteiga
#212121	#EAEAEA	#B86B84	#E0B94F	#DADADA	#FFDF7F

O rosa é a cor das ações principais (comprar, pagar), a mostarda e a manteiga destacam preços, avaliações e áreas de boas-vindas, e a tinta dá contraste. As capas dos livros são geradas com essas seis cores em doze combinações, sem usar nenhuma imagem de terceiros.

CAPÍTULO 2

Como rodar

Com Docker, o Prelo sobe com um comando. Os scripts criam o arquivo de configuração com senhas aleatórias, constroem as imagens e abrem o navegador quando tudo estiver pronto.

Requisitos

- **Docker** com Docker Compose (Docker Desktop no Windows e no macOS).
- Cerca de 3 GB de memória livre (4 GB se for usar Oracle ou SQL Server) e 3 GB de disco para as imagens.
- Na primeira vez, acesso à internet para baixar as imagens e as dependências do Maven e do npm.

Primeira execução

Windows

Dê dois cliques em `iniciar.cmd`, na pasta do projeto.

Linux ou macOS

```
./iniciar.sh
```

Os dois scripts fazem a mesma coisa:

1. Conferem se o Docker está instalado.
2. Na primeira vez, copiam `.env.example` para `.env` e trocam `DB_PASSWORD` e `JWT_SECRET` por valores aleatórios.
3. Ajustam `COMPOSE_PROFILES` para o banco escolhido em `DB_VENDOR`.
4. Rodam `docker compose up -d --build`: sobem o banco, o Redis, a API e a interface.
5. Esperam `/api/health` responder e abrem **http://localhost:8080**.

Endereço	O que é
http://localhost:8080	Loja e administração
http://localhost:8080/api/docs	Documentação interativa da API (Swagger)
http://localhost:8080/api/health	Saúde da API, do banco e do Redis (usada pelos scripts)

Escolher o banco

O banco é definido no `.env`. Os dois valores precisam ser iguais (os scripts corrigem `COMPOSE_PROFILES` sozinhos):

```
DB_VENDOR=sqlserver
COMPOSE_PROFILES=sqlserver
```

Valor	Imagem usada	Primeira subida
`postgresql`	postgres:16-alpine	segundos
`mysql`	mysql:8.4	cerca de 30 s
`sqlserver`	mssql/server:2022 (edição Developer)	1 a 2 min; um contêiner auxiliar cria o banco e o login
`oracle`	gvenzl/oracle-free:23-slim-faststart	1 a 3 min; o usuário da aplicação é criado no FREEPDB1

TROCAR DE BANCO

Cada banco guarda os dados no próprio volume. Trocar `DB_VENDOR` começa uma loja nova (com os dados de demonstração, se ligados); voltar ao banco anterior traz os dados dele de volta.

Contas de demonstração

Num banco vazio, com `APP_SEED_DEMO=true`, o Prelo cria 38 livros, 8 categorias, 5 catálogos, 3 cupons, quatro contas e seis pedidos em etapas diferentes (entregue, a caminho, saiu para entrega, em separação e aguardando boleto). Todos os nomes, e-mails, endereços, editoras, preços e ISBNs são fictícios.

Papel	E-mail	Senha	O que pode fazer
Cliente	`leitora@prelo.dev`	`Prelo@2026`	Comprar, pagar, acompanhar pedidos, gerenciar endereços
Administração	`admin@prelo.dev`	`Prelo@2026`	Tudo, incluindo painel, pedidos, envios, livros e vitrines

Cupons de demonstração: `BEMVINDO10` (10%), `LEIA20` (R$ 20 acima de R$ 150) e `CLASSICOS15` (15% acima de R$ 100).

Comandos do dia a dia

Para	Comando
Ver os logs da API	`docker compose logs -f api`
Parar (os dados ficam nos volumes)	`docker compose down`
Atualizar depois de baixar uma versão nova	`docker compose up -d --build`
Abrir o terminal do banco (PostgreSQL)	`docker compose exec postgresql psql -U prelo`
Abrir o terminal do Redis	`docker compose exec redis redis-cli`
Apagar tudo, inclusive os dados	`docker compose down -v`

Usar de verdade

1. No `.env`, defina `APP_SEED_DEMO=false` antes da primeira subida (ou apague os volumes com `docker compose down -v`).
2. Se não for testar pagamentos simulados, defina `PAYMENTS_SANDBOX=false` e implemente um provedor real (capítulo 5).
3. Suba o projeto e crie a sua conta pela tela **Criar conta**.
4. Promova a sua conta a administração direto no banco (o comando é o mesmo nos quatro):

```
UPDATE app_user SET user_role = 'ADMIN' WHERE email = 'voce@sualivraria.com.br';
```

Depois, saia e entre de novo para o novo papel valer no token.

Rodar sem Docker

Útil para desenvolver. Você precisa de Java 21, Maven 3.9, Node 22, Redis 7 e um dos quatro bancos com um banco `prelo` e um usuário `prelo`.

```
# API na porta 8081 (PostgreSQL local por padrão)
cd backend
DB_PASSWORD=prelo JWT_SECRET=troque-por-uma-chave-com-32-caracteres mvn spring-boot:run

# outro banco
DB_VENDOR=mysql DB_HOST=127.0.0.1 DB_PASSWORD=prelo mvn spring-boot:run

# interface na porta 5173 (repassa /api para a 8081)
cd frontend && npm install && npm run dev
```

Acesso pela rede e HTTPS

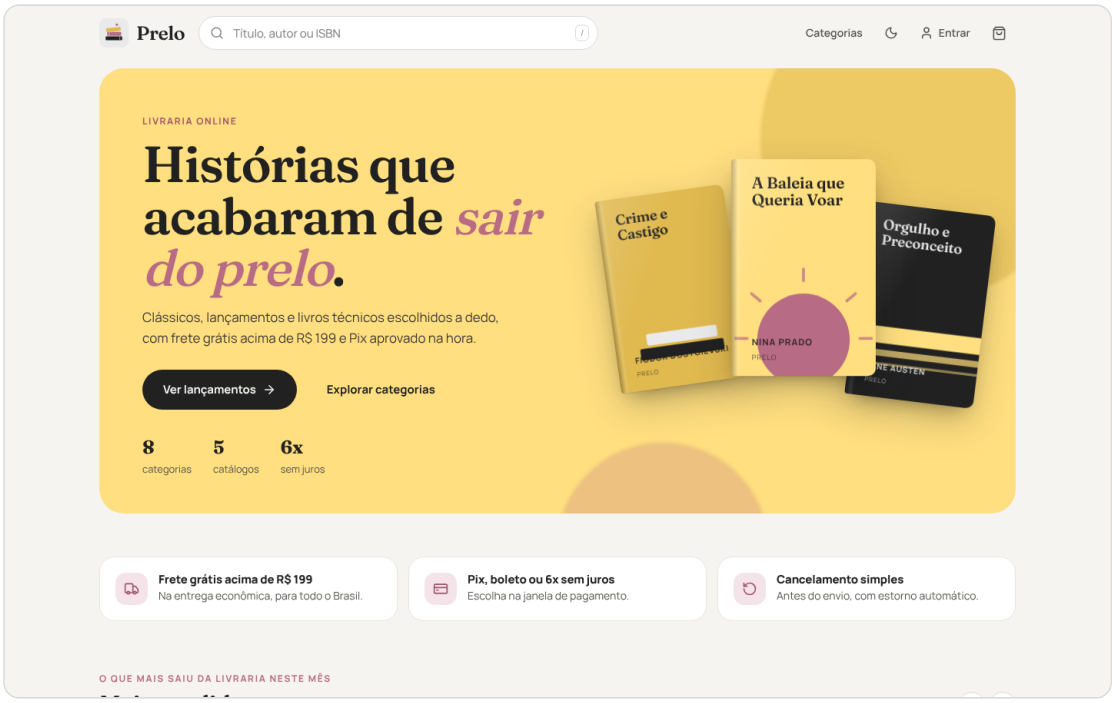
Variável	Quando mexer
`APP_PORT`	Para publicar em outra porta (padrão 8080).
`APP_HTTPS`	`true` atrás de um proxy com HTTPS: o cookie de sessão passa a ser só HTTPS.
`APP_CORS_ORIGINS`	Endereços que podem chamar a API de outro domínio (a interface própria não precisa).
`API_DOCS_ENABLED`	`false` desliga o Swagger em produção.

CAPÍTULO 3

Guia das telas

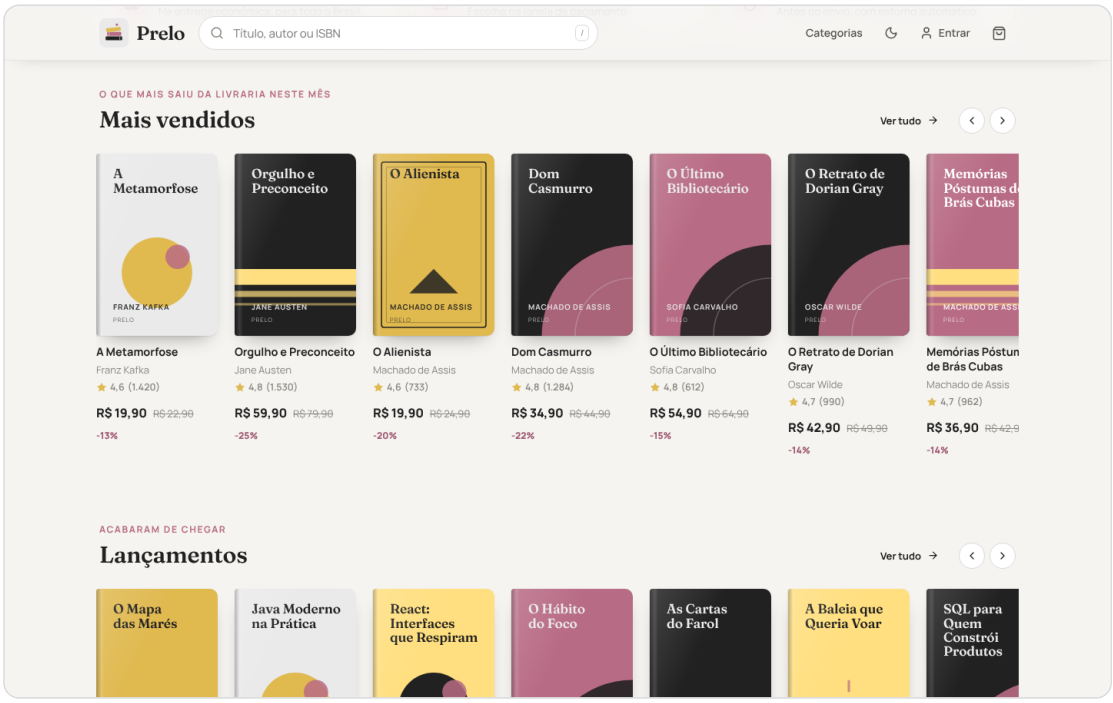
Cada tela, o que ela mostra e como usar. Todas funcionam no tema claro e no escuro e se adaptam ao celular.

Vitrine



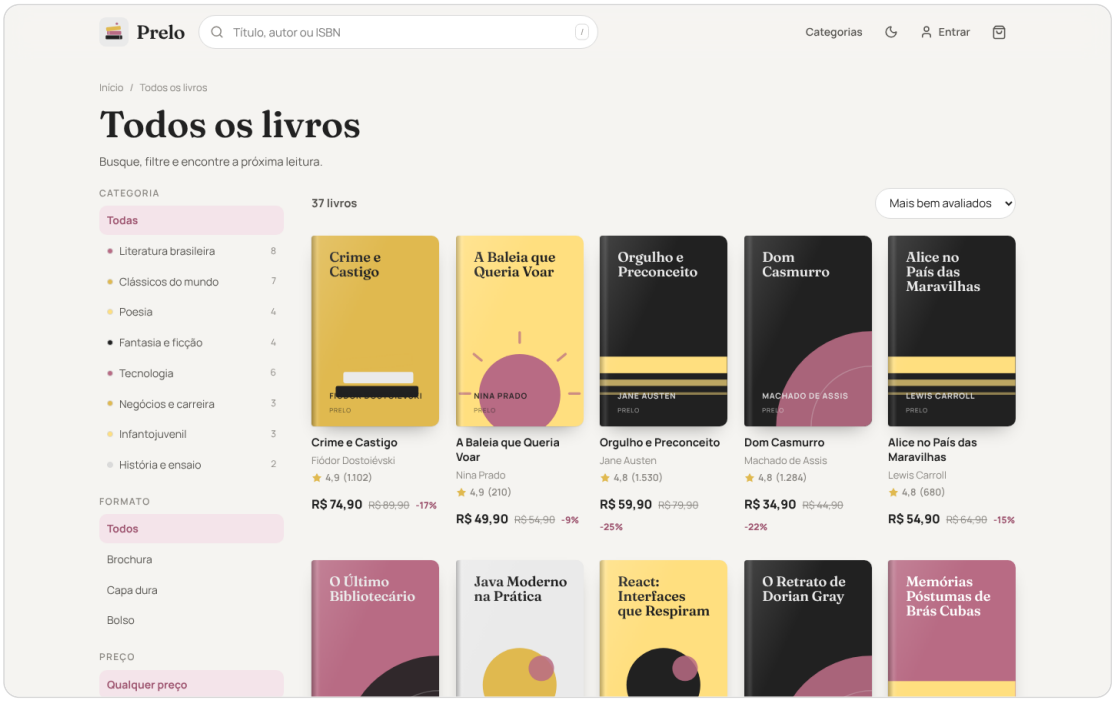
Vitrine com destaque, benefícios e os primeiros catálogos

- **Destaque:** três livros mais bem avaliados entre os mais vendidos, flutuando ao lado da chamada. Clique em um para abrir a página dele.
- **Benefícios:** frete grátis acima de R$ 199, formas de pagamento e cancelamento simples.
- **Catálogos em trilhos:** cada catálogo curado vira um trilho horizontal. Use as setas, arraste no celular ou clique em **Ver tudo**.
- **Categorias e cupom:** blocos coloridos levam a cada categoria; a faixa escura mostra o cupom de primeira compra.
- **Busca:** a caixa no topo está em todas as telas. Aperte `/` para ir direto a ela.



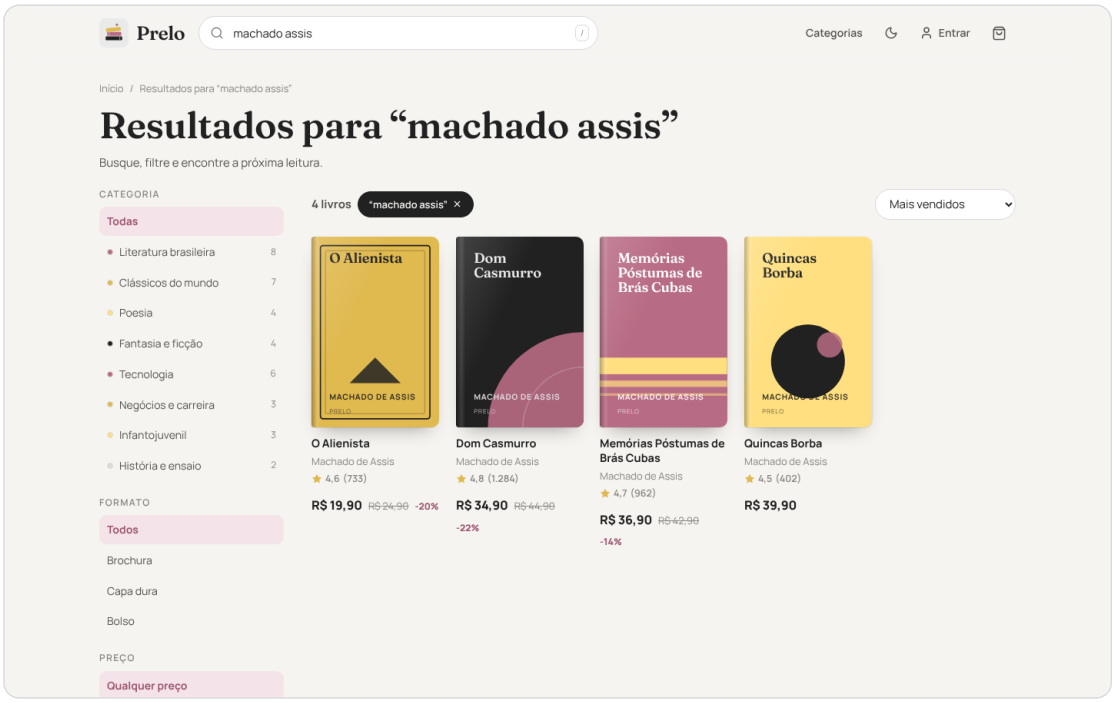
Trilhos dos catálogos: o botão + adiciona o livro ao carrinho sem sair da vitrine

Catálogo e busca



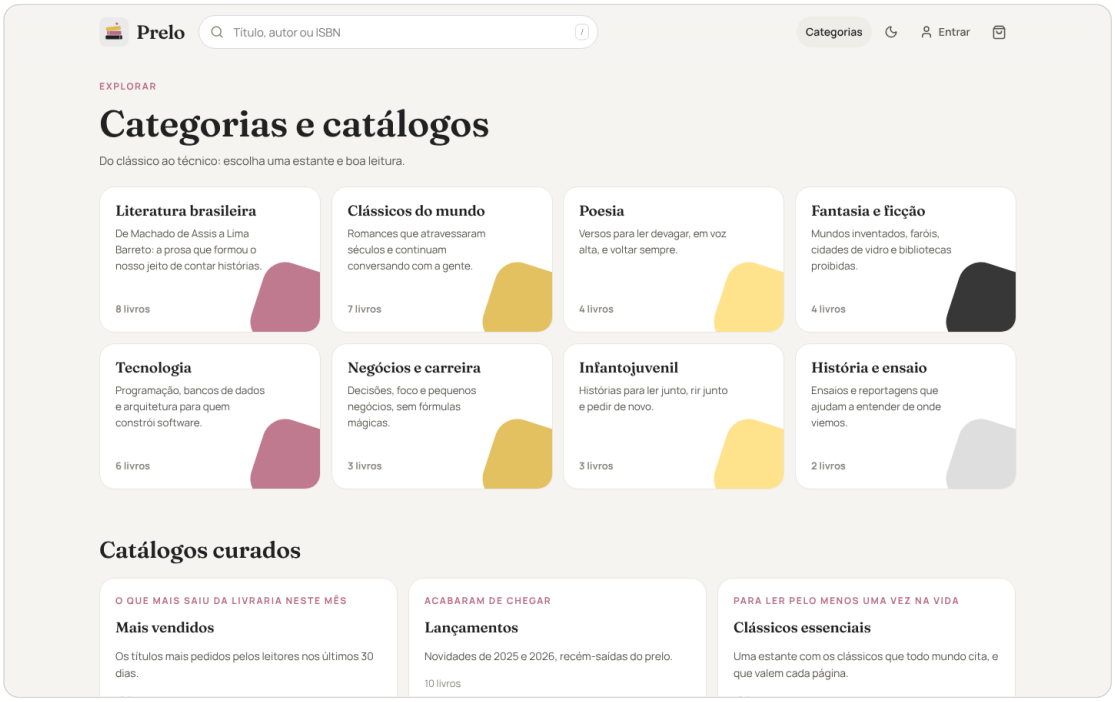
Todos os livros com filtros à esquerda e ordenação

A lista de livros reúne os filtros à esquerda (categoria, formato, faixa de preço e só o que tem em estoque) e a ordenação à direita: mais vendidos, novidades, mais bem avaliados, menor preço, maior preço e título. Cada filtro ativo vira um selo que se remove com um clique. A busca ignora acentos e maiúsculas: memorias postumas encontra Memórias Póstumas de Brás Cubas, e o ISBN funciona com ou sem hífens.



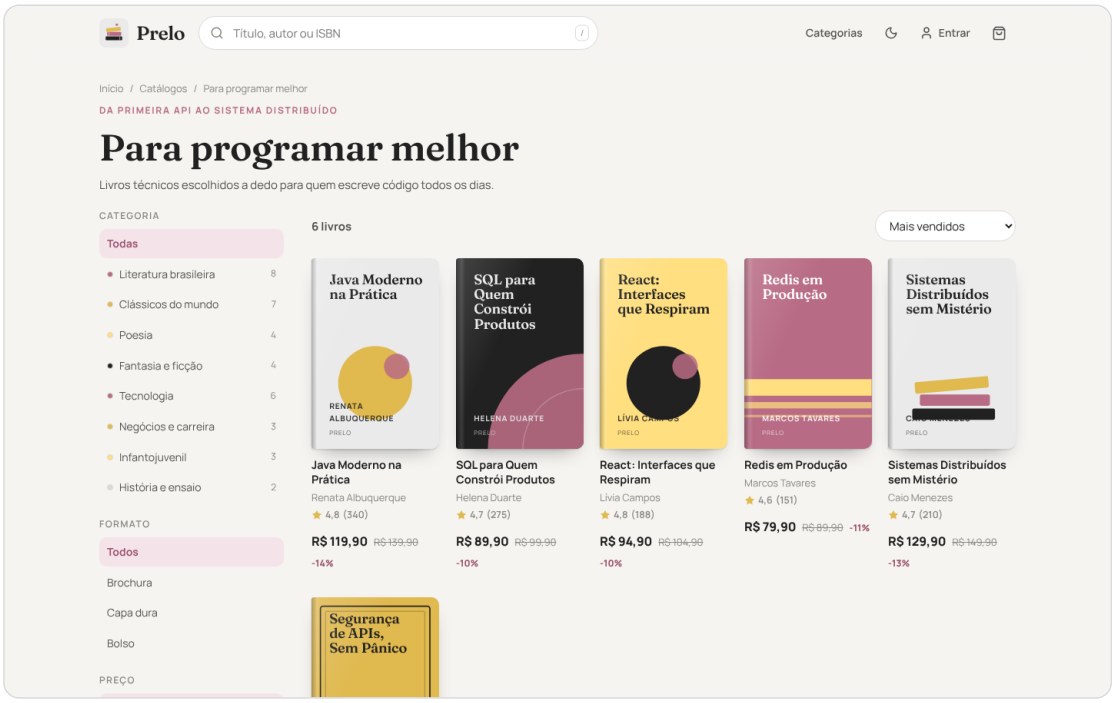
Busca por autor

Categorias e catálogos



Categorias com descrição e contagem, e os catálogos curados

A página **Categorias** mostra as oito estantes com descrição e número de livros, e embaixo os catálogos curados. Um catálogo tem ordem própria, definida pela livraria; abrir um catálogo mostra os livros nessa ordem, com os mesmos filtros da busca.



Catálogo curado Para programar melhor

Página do livro

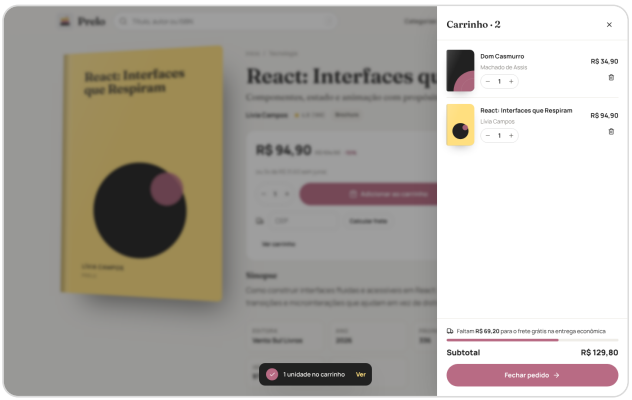


Página do livro com preço, estoque, frete e ficha técnica

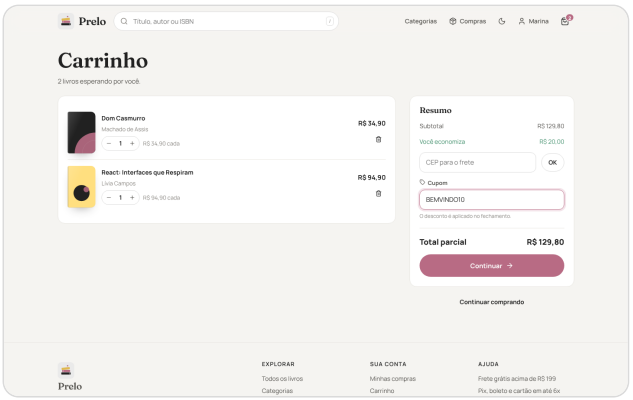
- **Preço:** o de venda, o de capa riscado e o desconto. Abaixo, as parcelas sem juros (parcela mínima de R$ 30).
- **Estoque:** verde quando há bastante, mostarda com Restam só N quando sobram 5 ou menos, vermelho quando esgotado.
- **Quantidade e compra:** escolha a quantidade (até 10 ou o estoque) e use **Adicionar ao carrinho** ou **Comprar agora**, que já leva ao carrinho.

- **Frete:** digite o CEP e veja preço e prazo das três formas de envio para essa quantidade.
- **Ficha técnica e sugestões:** editora, ano, páginas, idioma, ISBN e peso; embaixo, livros da mesma categoria.

Carrinho



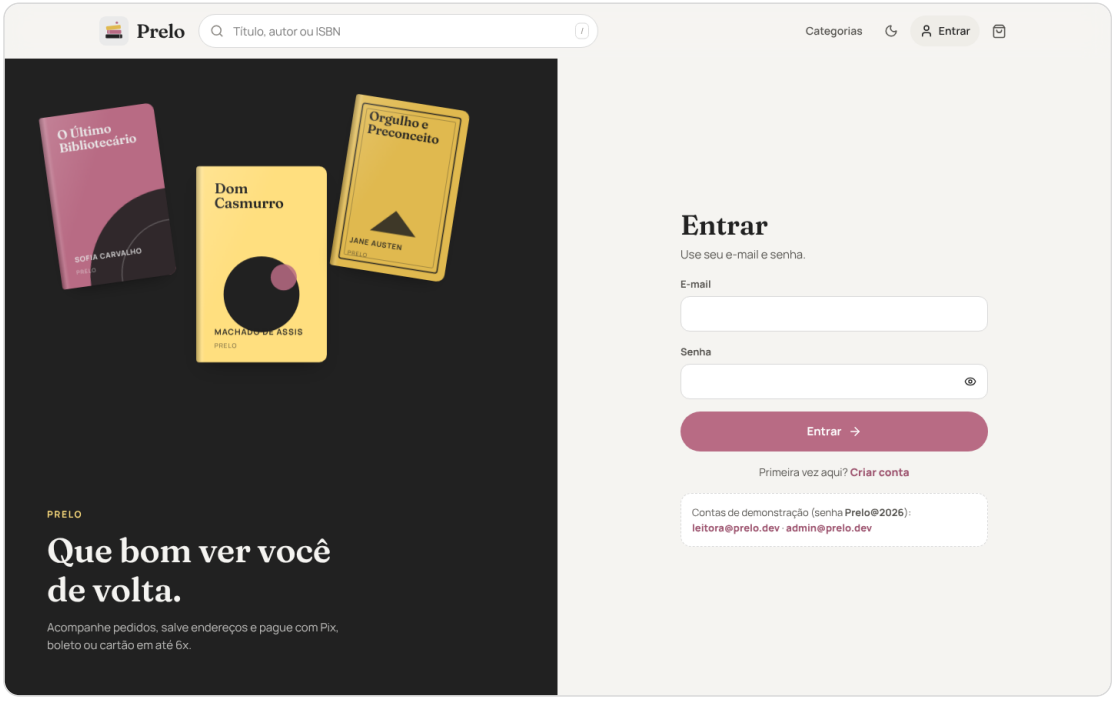
Gaveta do carrinho com a barra de frete grátis



Carrinho com resumo, frete e cupom

O ícone da sacola abre a **gaveta** com os itens, a barra que mostra quanto falta para o frete grátis e o botão **Fechar pedido**. A página **Carrinho** tem o resumo completo, a economia em relação ao preço de capa, a estimativa de frete e o campo de cupom. Sem login, o carrinho fica no navegador; ao entrar, ele é somado ao da conta (respeitando o estoque e o limite de 10 por título).

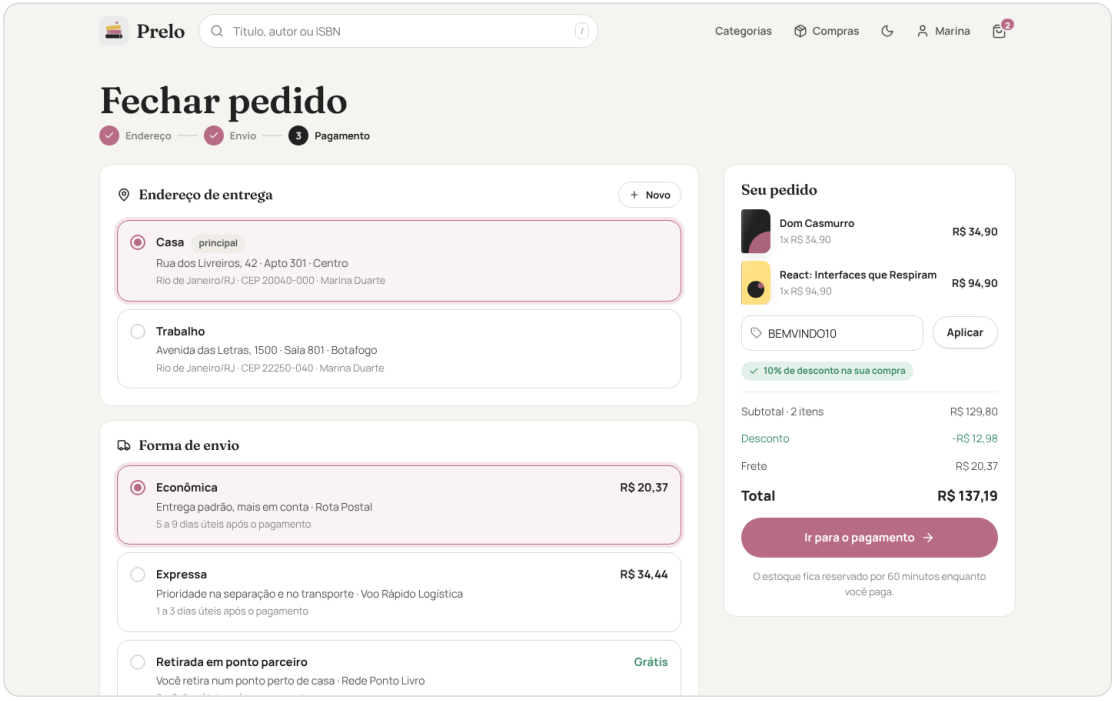
Entrar e criar conta



Tela de entrada com as contas de demonstração

A entrada pede e-mail e senha; o link **Criar conta** pede também o nome. Na tela de entrada, os e-mails de demonstração preenchem o formulário com um clique. Depois de cinco senhas erradas seguidas, o e-mail fica bloqueado por 15 minutos.

Checkout

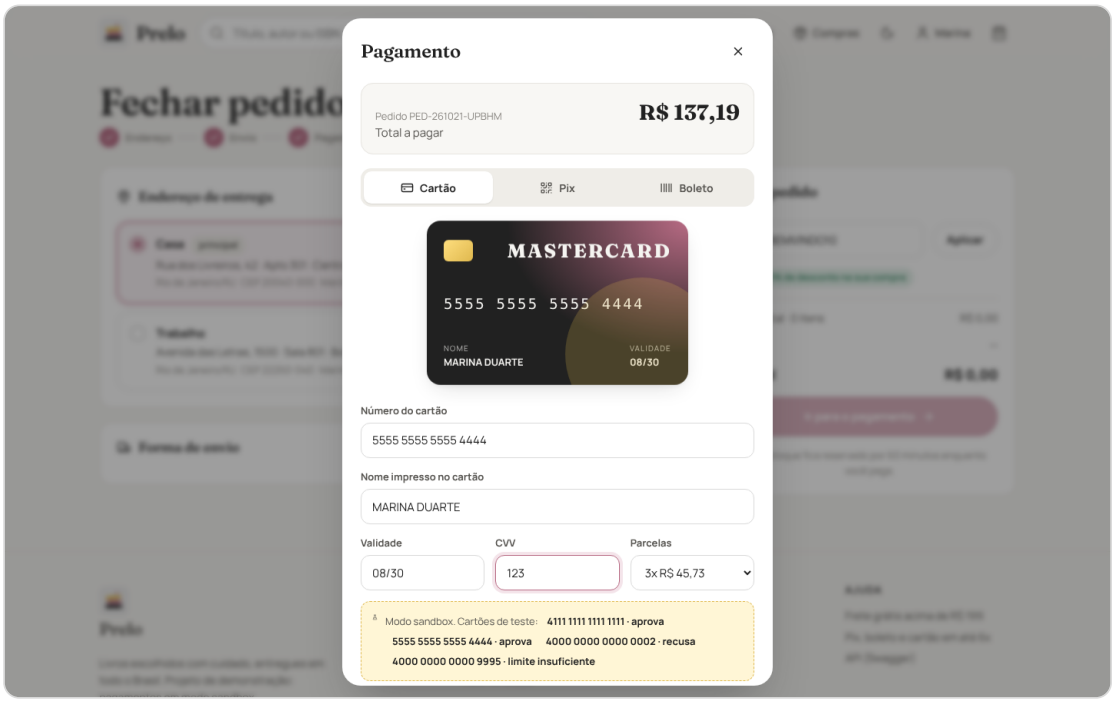


Checkout: endereço, forma de envio e resumo com cupom

1. **Endereço:** escolha um dos seus endereços ou crie um novo. Ao digitar o CEP, rua, bairro, cidade e UF são preenchidos automaticamente (quando a consulta de CEP está disponível).
2. **Envio:** Econômica (grátis acima de R$ 199), Expressa ou Retirada em ponto parceiro (grátis acima de R$ 99). Preço e prazo mudam com a região do CEP e o peso.
3. **Resumo:** confira os itens, aplique o cupom e veja o total. **Ir para o pagamento** cria o pedido, reserva o estoque por 60 minutos e abre a janela de pagamento.

Janela de pagamento

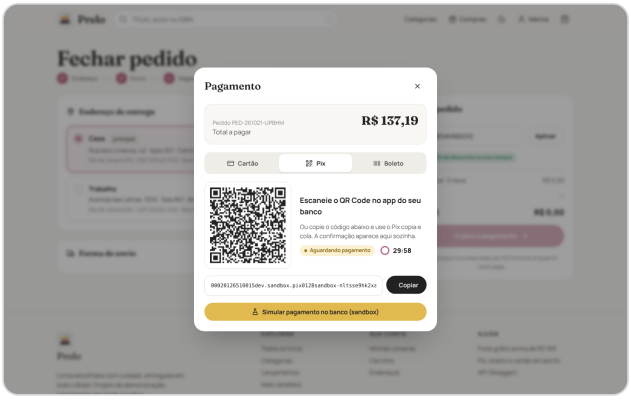
A janela mostra o total e três abas. Ela pode ser fechada a qualquer momento: o pedido fica em **Aguardando pagamento** e o botão **Pagar** aparece de novo em Minhas compras.



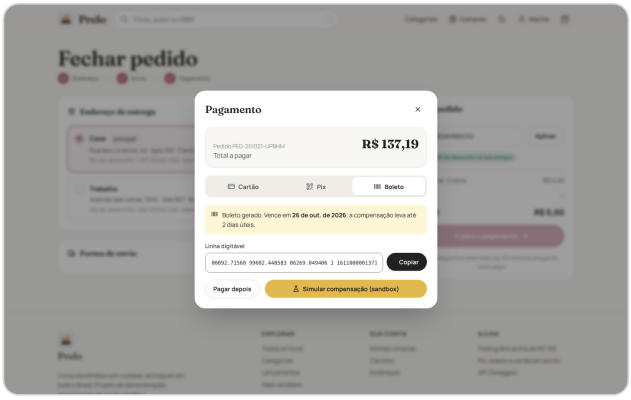
Cartão de crédito com parcelas e os cartões de teste do sandbox

Cartão de crédito

O cartão desenhado acompanha o que você digita e reconhece a bandeira (Visa, Mastercard, Elo, Amex, Hipercard). Escolha as parcelas e clique em **Pagar**. Se o emissor recusar, o motivo aparece em vermelho e você pode tentar outro cartão sem perder o pedido.



Pix com QR Code, copia e cola e contagem regressiva



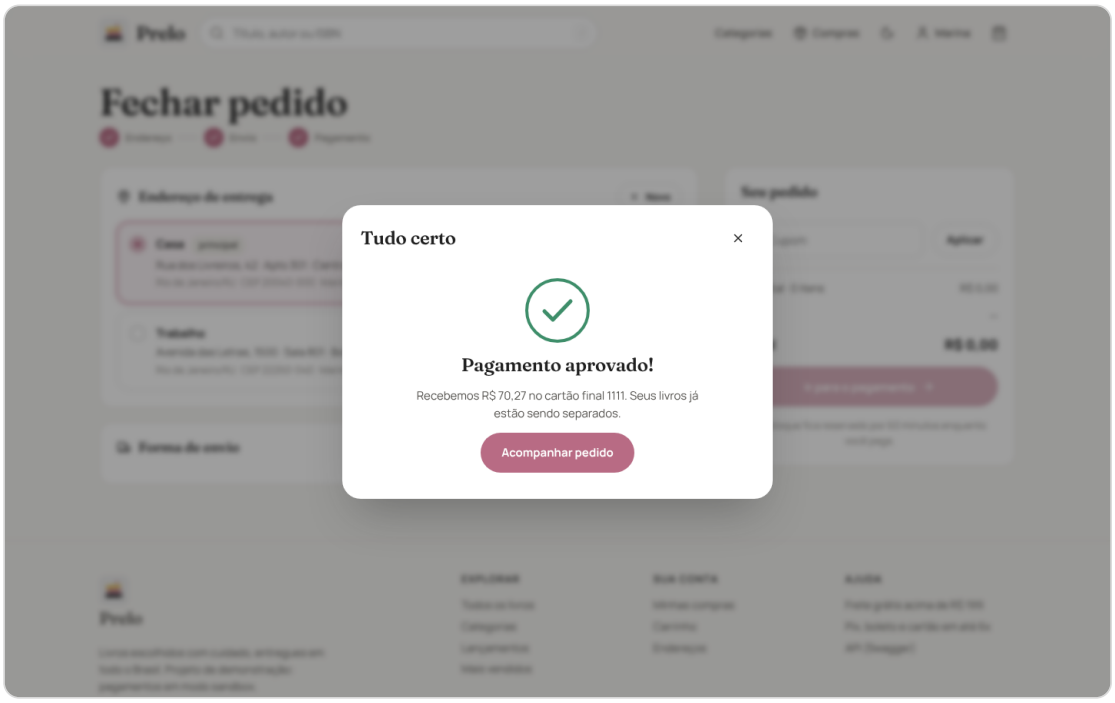
Boleto com linha digitável e vencimento

Pix

Gerar QR Code Pix mostra o QR Code, o código copia e cola e um relógio de 30 minutos. A janela consulta o pagamento a cada 3 segundos e passa sozinha para a confirmação quando ele cai.

Boleto

Gerar boleto mostra a linha digitável e o vencimento (3 dias úteis). O prazo do pedido se estende até o vencimento, e os livros saem depois da compensação. Pix e boleto podem existir ao mesmo tempo; quando um é pago, o outro deixa de valer.

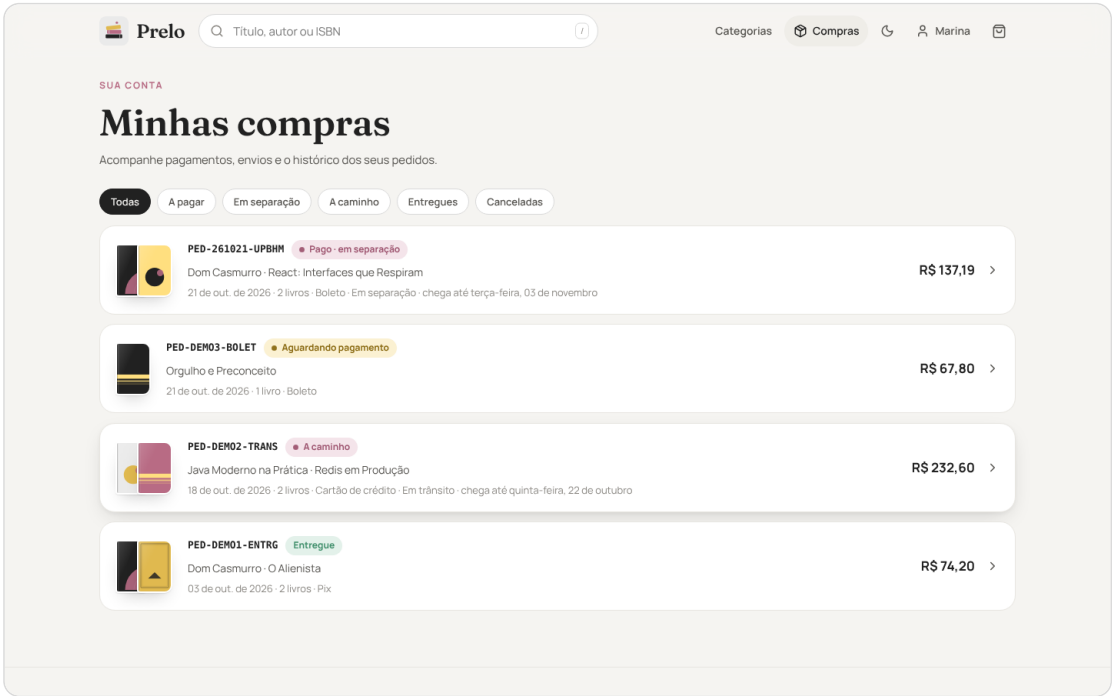


Pagamento aprovado

SANDBOX

Com `PAYMENTS_SANDBOX=true`, a aba do cartão lista os cartões de teste (clique para preencher) e Pix e boleto ganham o botão **Simular pagamento**, que faz o papel do banco.

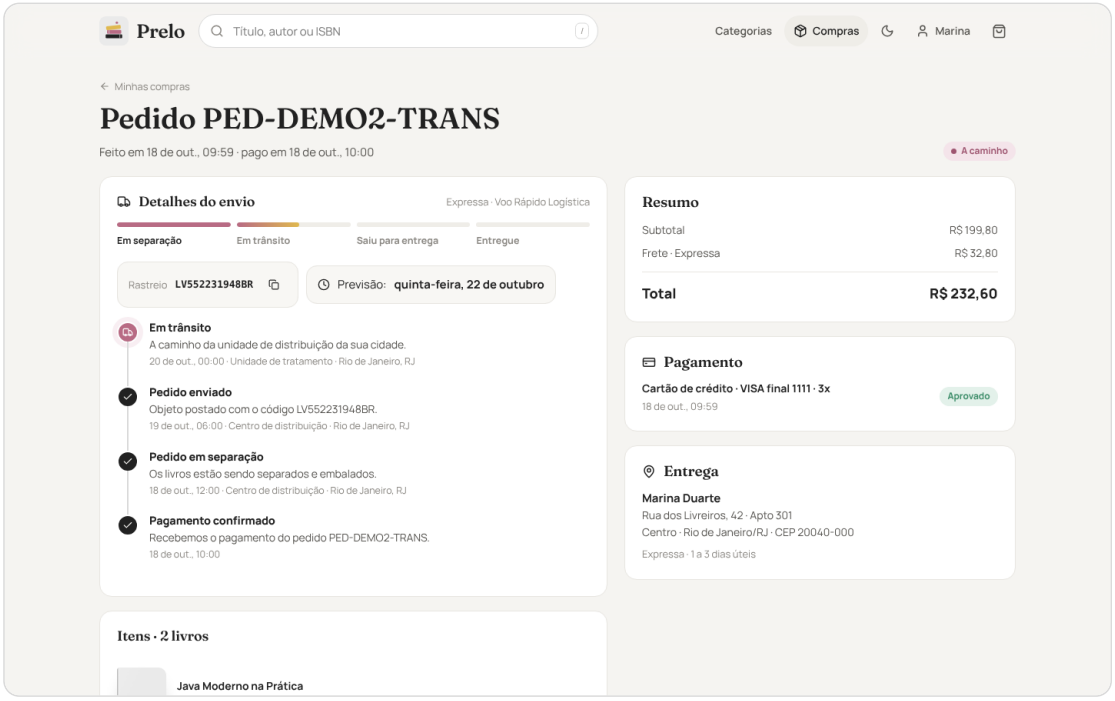
Minhas compras



Minhas compras com filtros por status

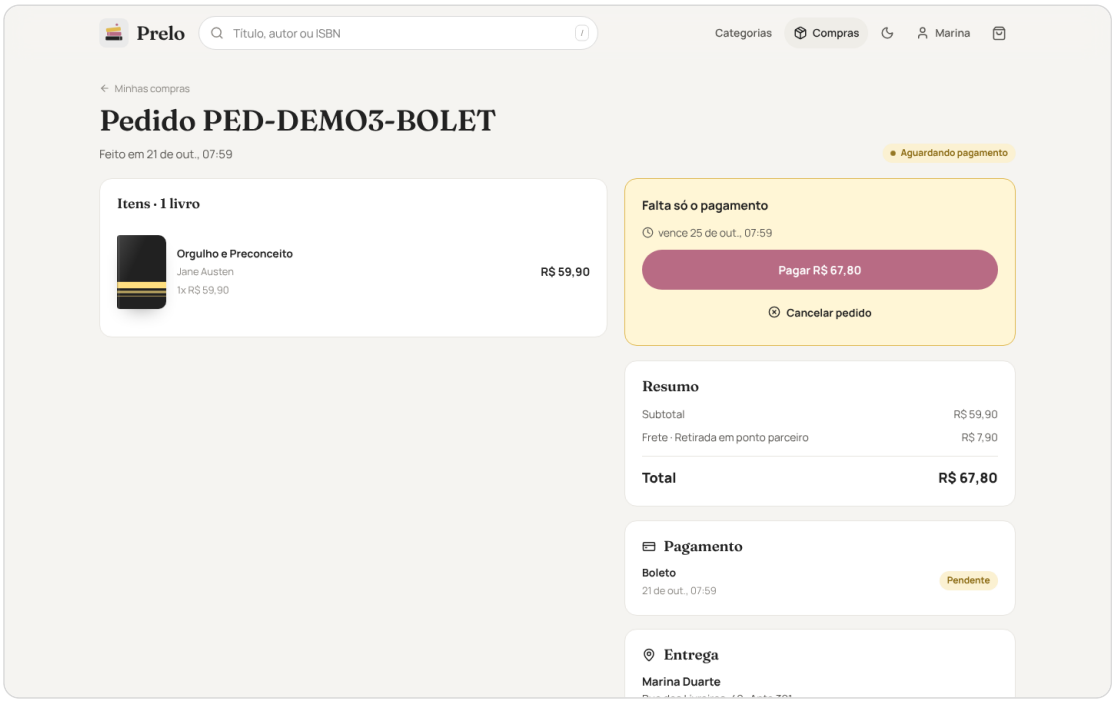
A aba **Compras** lista os pedidos do mais novo para o mais antigo, com as capas, o status, a forma de pagamento, a etapa do envio e a previsão de chegada. Os filtros separam o que está a pagar, em separação, a caminho, entregue ou cancelado.

Detalhes do pedido e do envio



Pedido a caminho: progresso, rastreio, previsão e linha do tempo

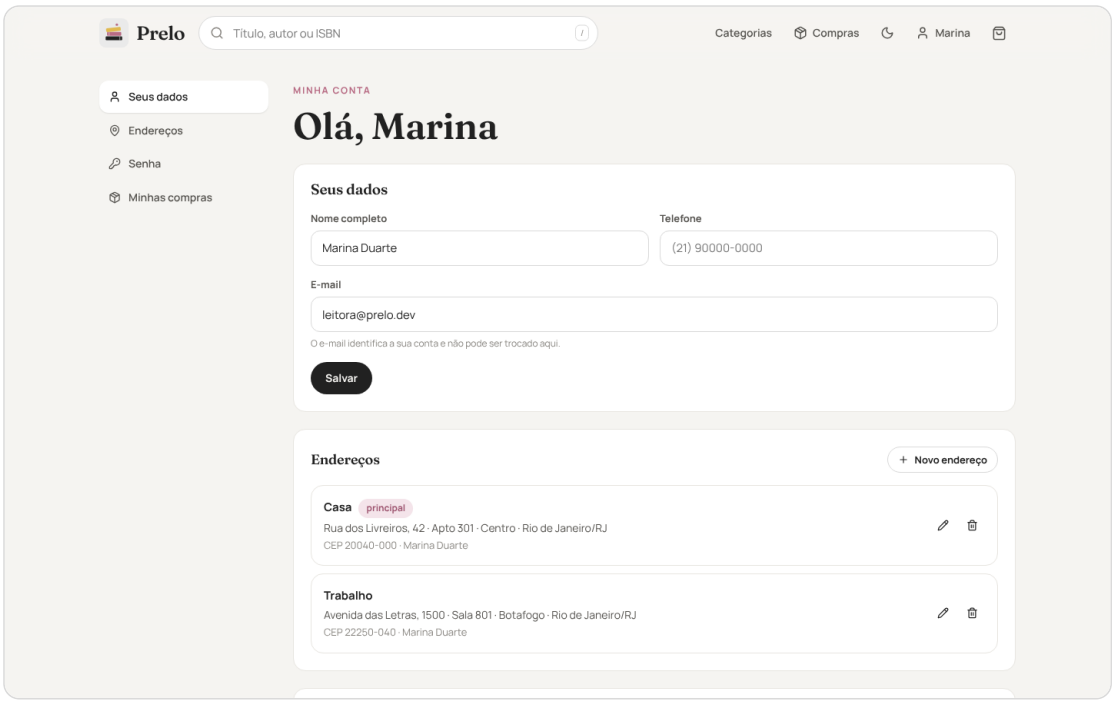
- **Detalhes do envio:** barra com as quatro etapas (em separação, em trânsito, saiu para entrega, entregue), código de rastreio com botão de copiar, previsão de entrega em dias úteis e a linha do tempo com data, hora e local de cada evento.
- **Itens:** cada livro com quantidade e preço pagos (o preço fica guardado no pedido mesmo que mude depois).
- **Resumo, pagamento e entrega:** subtotal, desconto, frete e total; todas as tentativas de pagamento com o status de cada uma; o endereço copiado no momento da compra.



Pedido aguardando pagamento, com o tempo restante e o botão de cancelar

Enquanto o pedido não foi enviado, **Cancelar pedido** devolve os livros ao estoque e estorna o que já foi pago. Pedidos sem pagamento até o fim do prazo são cancelados automaticamente.

Minha conta



Dados, endereços e troca de senha

Edite nome e telefone, gerencie até 10 endereços (um deles é o principal, usado primeiro no checkout) e troque a senha informando a atual. O e-mail identifica a conta e não muda por aqui.

Tema escuro e celular



Vitrine no tema escuro

O tema segue o sistema e pode ser trocado pelo ícone da lua ou do sol no topo; a escolha fica salva no navegador. No celular, a busca encolhe, os filtros viram um botão e os trilhos deslizam com o dedo.



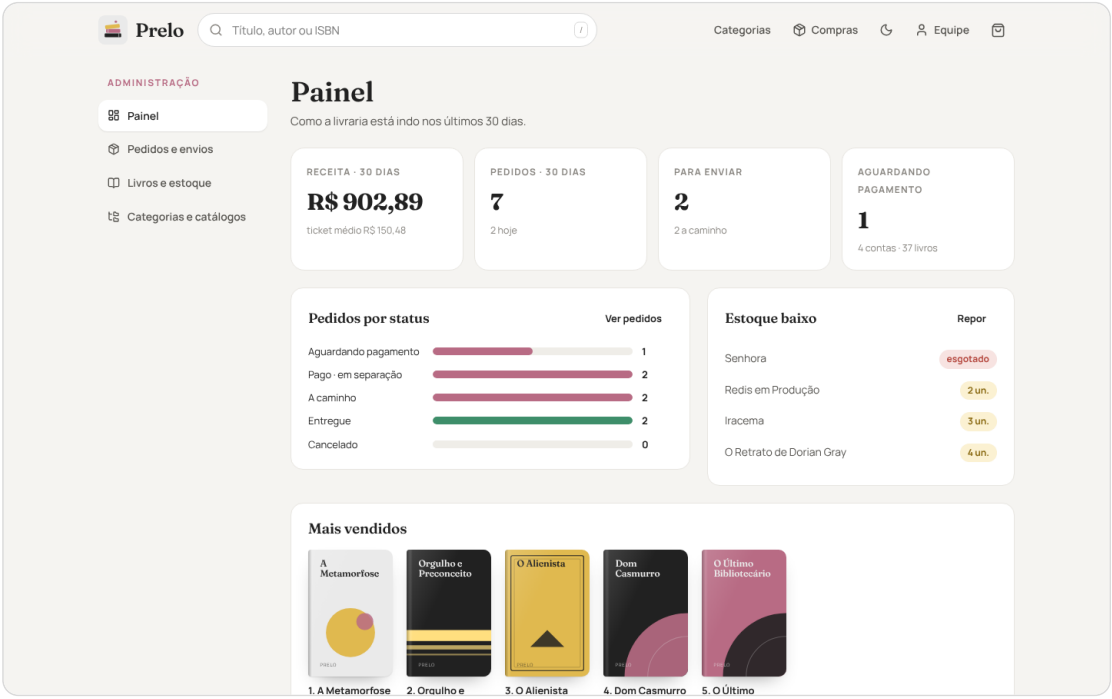
Vitrine, livro e catálogo no celular

CAPÍTULO 4

Administração

A área da livraria aparece no menu da conta para quem tem o papel de administração.

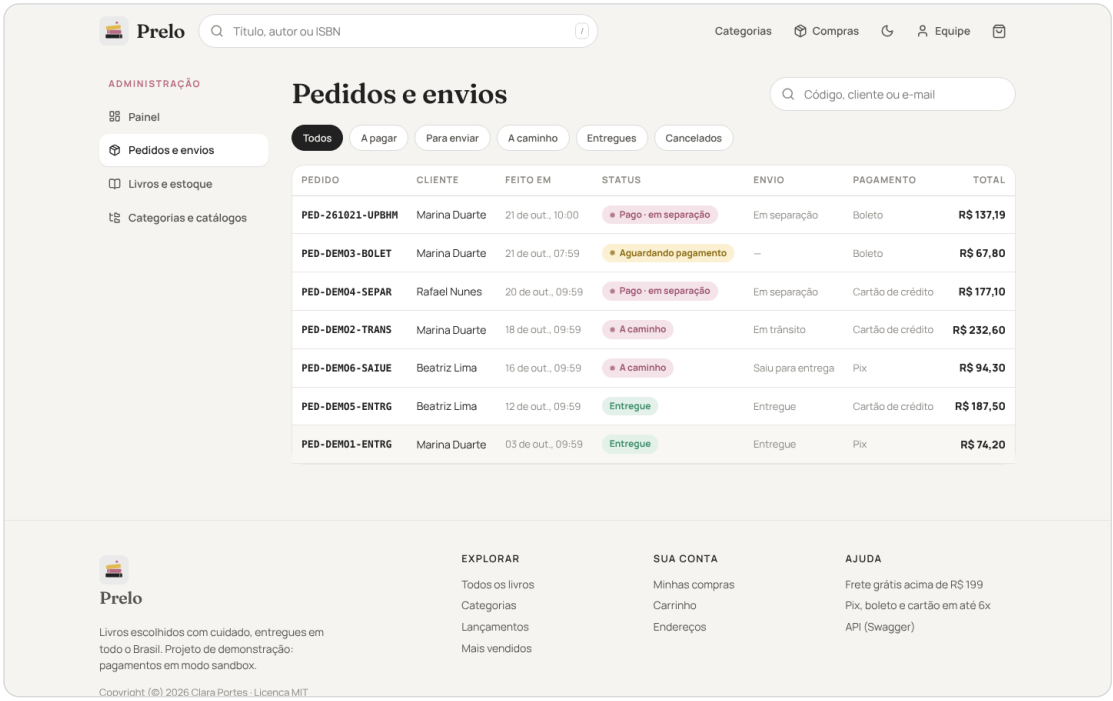
Painel



Painel com receita, pedidos por status, estoque baixo e mais vendidos

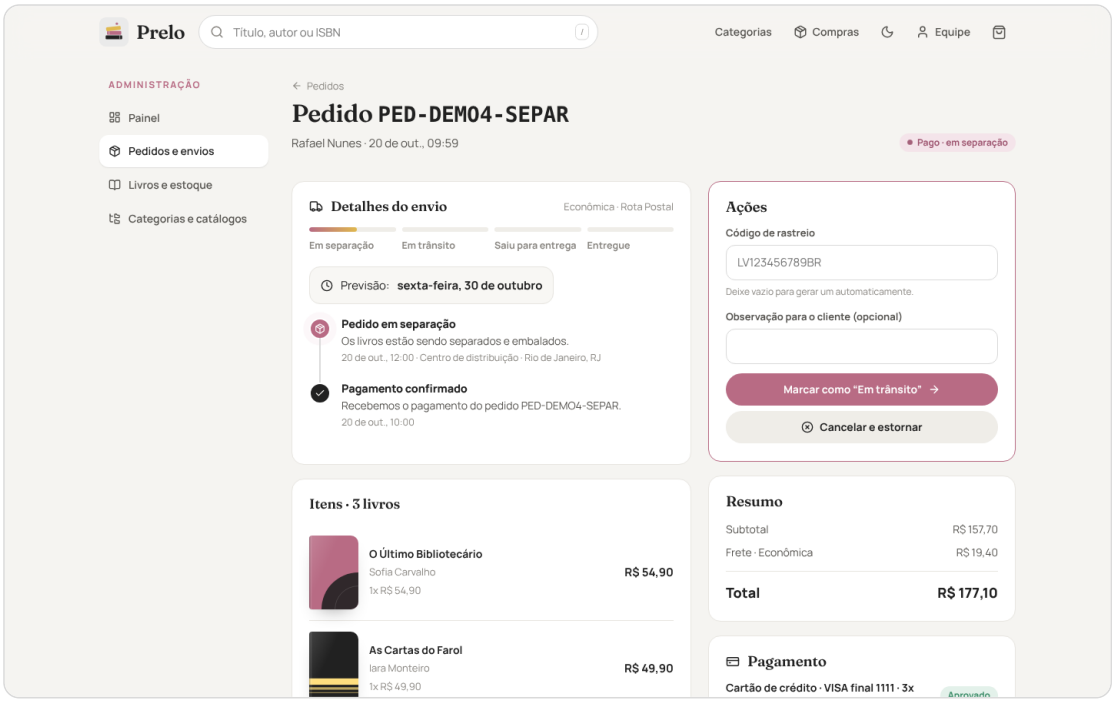
- **Receita e ticket médio** dos pedidos pagos nos últimos 30 dias (cancelados ficam de fora).
- **Pedidos** nos últimos 30 dias e hoje; **para enviar** (em separação) e **a caminho**; **aguardando pagamento**.
- **Pedidos por status:** barras que levam à lista filtrada.
- **Estoque baixo:** livros com 5 unidades ou menos, com atalho para repor; **mais vendidos** com as capas.

Pedidos e envios



Pedidos com filtros e busca por código, cliente ou e-mail

A lista mostra todos os pedidos com cliente, data, status, etapa do envio, forma de pagamento e total. Busque pelo código, pelo nome de quem recebe ou pelo e-mail exato da conta.

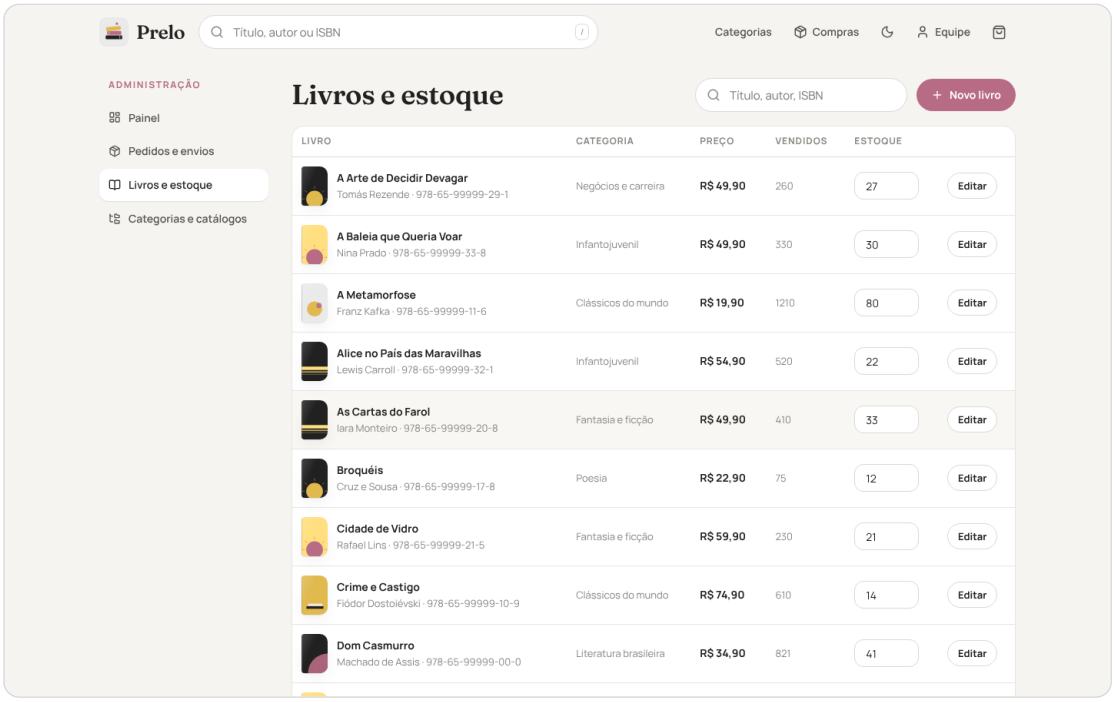


Pedido em separação com o painel de ações

1. Abra um pedido pago. O painel **Ações** mostra a próxima etapa do envio.
2. Em **Marcar como Em trânsito**, informe o código de rastreio da transportadora ou deixe vazio para gerar um. O pedido passa para **A caminho**.

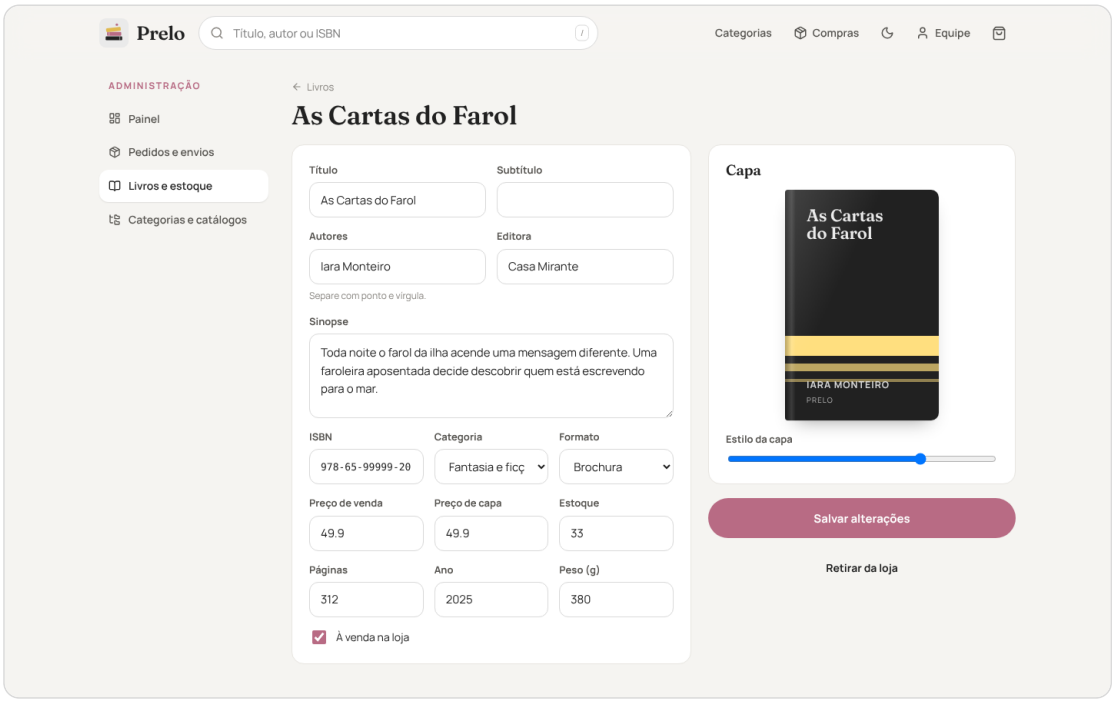
3. Depois, **Saiu para entrega** e **Entregue**. Cada clique cria um evento na linha do tempo do cliente; a observação opcional aparece para ele.
4. **Cancelar e estornar** fica disponível enquanto o pedido não foi postado.

Livros e estoque



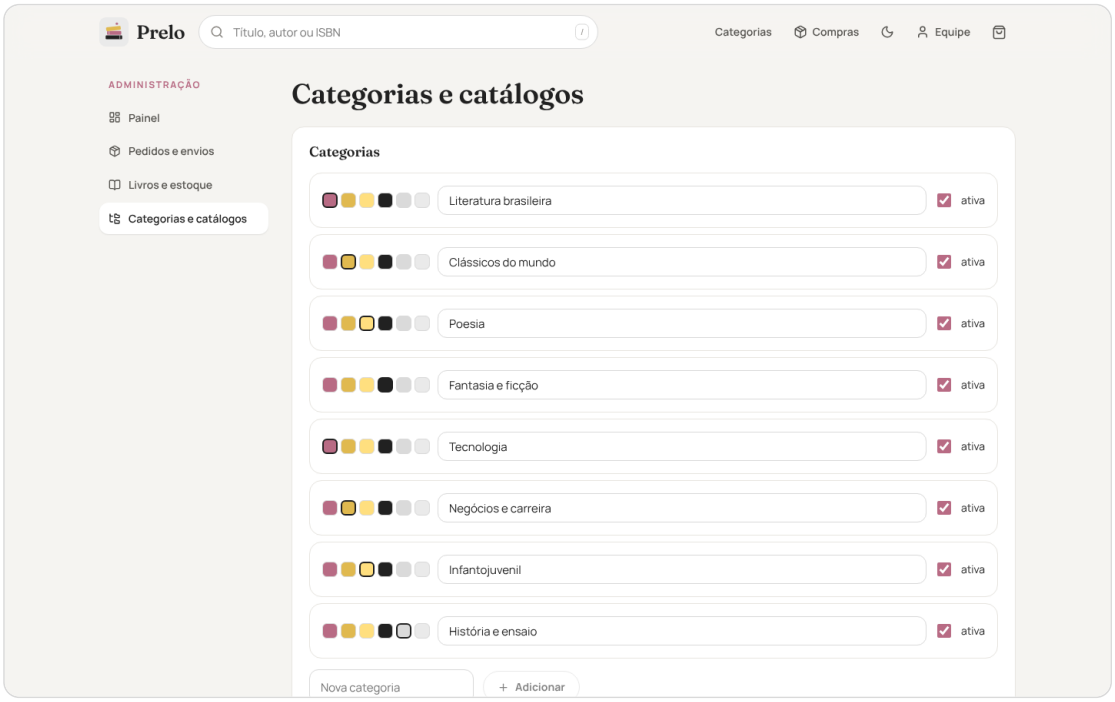
Livros com estoque editável direto na lista

A lista mostra preço, vendidos e estoque. Mude o número do estoque e confirme com Enter ou com o botão que aparece. **Novo livro** e **Editar** abrem o formulário completo: título, subtítulo, autores (separados por ponto e vírgula; autores e editoras novos são criados na hora), editora, sinopse, ISBN, categoria, formato, preços, estoque, páginas, ano, peso e o estilo da capa, com prévia ao vivo. **Retirar da loja** esconde o livro sem apagar o histórico de pedidos.



Edição de livro com a prévia da capa

Categorias e catálogos



Categorias com cor e catálogos curados

Renomeie categorias, troque a cor (uma das seis da paleta) e ative ou desative cada uma. Crie catálogos novos e escolha quais aparecem na vitrine. A ordem dos livros de cada catálogo pode ser definida pela API (`PUT /api/admin/catalogs/{id}` com a lista de ids).

CAPÍTULO 5

Pagamentos e envio

As regras de dinheiro e de entrega ficam na API; a interface só mostra o que a API decide.

Sandbox e cartões de teste

O provedor que vem com o projeto é um sandbox: nada sai da aplicação e nenhum código gerado paga algo de verdade. O Pix usa um identificador de sandbox (não é um BR Code válido em bancos) e o boleto usa o banco fictício 000.

Cartão	Resultado
`4111 1111 1111 1111`	Aprovado (Visa)
`5555 5555 5555 4444`	Aprovado (Mastercard)
`4000 0000 0000 0002`	Recusado pelo emissor
`4000 0000 0000 9995`	Limite insuficiente
Qualquer outro número válido	Aprovado

Qualquer nome, validade futura e CVV de 3 dígitos (4 no Amex) funcionam.

Como o cartão é protegido

1. A janela envia número, nome, validade e CVV para `POST /api/payments/card-tokens`.
2. A API confere o dígito verificador (Luhn), a validade e a bandeira, e guarda no Redis só um token de 10 minutos com bandeira, 4 últimos dígitos e o resultado previsto. Número e CVV não são gravados em lugar nenhum nem aparecem nos logs.
3. O pagamento usa o token, que vale uma vez só. No banco fica a bandeira, os 4 últimos dígitos e as parcelas.

Idempotência

A janela manda um cabeçalho `Idempotency-Key` a cada tentativa. Se a mesma chave chegar de novo (clique duplo, conexão que caiu), a API devolve o mesmo pagamento em vez de criar outro. Pix e boleto pendentes do mesmo pedido também são reaproveitados enquanto valerem. O pedido tem bloqueio otimista: dois pagamentos ao mesmo tempo nunca aprovam o pedido duas vezes.

Plugar um provedor real

Implemente a interface `PaymentGateway` (`authorizeCard` , `pixPayload` , `boletoLine` , `newReference`) chamando o provedor escolhido, registre-a como bean no lugar do `SandboxGateway` , crie uma rota de webhook que chame a confirmação do pagamento e defina `PAYMENTS_SANDBOX=false` , que desliga os botões de simulação e a rota `/api/sandbox` .

Frete

As formas de envio ficam na tabela `shipping_method` (taxa base, valor por quilo, frete grátis a partir de, prazo mínimo e máximo). O peso é arredondado para cima em quilos e a região do CEP ajusta valor e prazo:

CEP começa com	Região	Fator no valor	Dias a mais
0 e 1	São Paulo	1,00	0
2	Rio de Janeiro e Espírito Santo	1,05	0
3	Minas Gerais	1,10	1
4	Bahia e Sergipe	1,30	2
5	PE, AL, PB e RN	1,40	3
6	Norte, CE, PI e MA	1,55	4
7	Centro-Oeste, TO, RO e AC	1,30	2
8	Paraná e Santa Catarina	1,15	1
9	Rio Grande do Sul	1,20	2

Prazos e cancelamento automático

- O estoque é reservado no checkout e o pedido tem **60 minutos** para ser pago (cartão e Pix). Gerar um boleto estende o prazo até o vencimento.
- A cada minuto, uma tarefa cancela os pedidos vencidos, marca os pagamentos pendentes como expirados e devolve o estoque. Uma trava no Redis garante que só uma instância da API faça isso por vez.
- A previsão de entrega conta dias úteis a partir do pagamento.

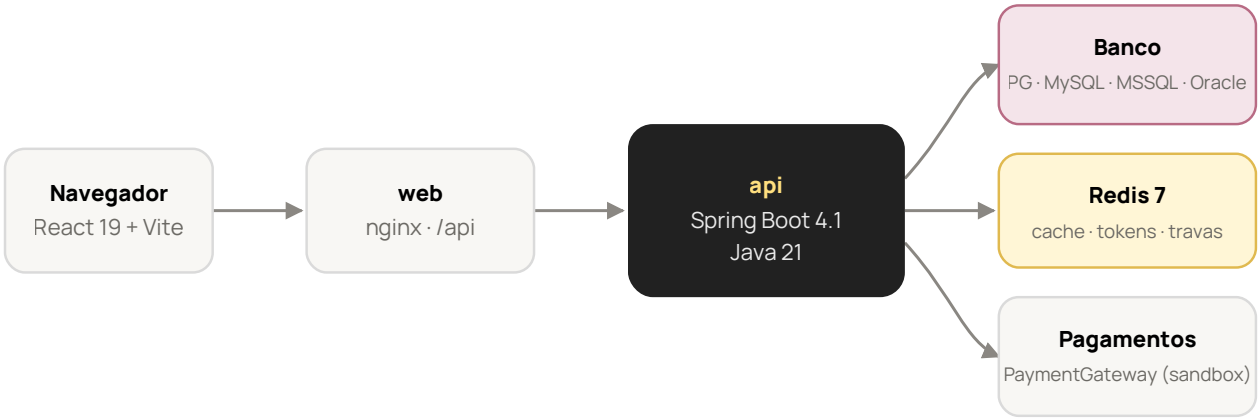
Cupons

Ficam na tabela `coupon`: percentual ou valor fixo, subtotal mínimo e validade. O frete grátis considera o subtotal já com o desconto.

CAPÍTULO 6

Arquitetura

Uma API Spring Boot organizada por domínio, uma interface React e dois serviços de dados: o banco relacional (qualquer um dos quatro) e o Redis.



Pacotes da API

Pacote	Responsabilidade
`auth`	Contas, login, cadastro, JWT, renovação de sessão no Redis, limite de tentativas
`account`	Perfil, senha e endereços
`catalog`	Livros, autores, editoras, categorias, catálogos curados, busca com Criteria e cache
`cart`	Carrinho da conta e junção com o carrinho do navegador
`order`	Checkout, cupons, pedidos, ciclo de vida (pago, enviado, entregue, cancelado) e expiração
`payment`	Tokens de cartão, gateway, cartão, Pix, boleto, idempotência e sandbox
`shipping`	Formas de envio, cotação por região e peso, envios e eventos de rastreio
`admin`	Painel e rotas de administração (pedidos, livros, estoque, categorias, catálogos)
`common` · `config`	Erros no formato Problem Details, normalização de texto, travas, segurança, cache, OpenAPI
`seed`	Formas de envio sempre; acervo, contas e pedidos de demonstração quando ligado

Segurança

- **Senhas** com BCrypt (custo 11). Login com e-mail inexistente gasta o mesmo tempo que um errado.
- **Token de acesso** JWT HS256 de 15 minutos, só na memória do navegador (nunca no localStorage).

- **Renovação** por cookie HttpOnly, SameSite Strict, restrito a `/api/auth`. O Redis guarda só o hash SHA-256 do token; cada uso gira o token e sair apaga a chave na hora.
- **Limite de tentativas:** 5 senhas erradas por e-mail em 15 minutos.
- **Autorização** por papel: `/api/admin/**` só para ADMIN; pedidos e pagamentos só aparecem para o dono (outra conta recebe 404).
- **Validação** de todas as entradas com Bean Validation e respostas de erro sem detalhes internos.
- **Cabeçalhos:** CSP, X-Frame-Options, nosniff e Referrer-Policy na API e no nginx.

Redis

Chave	Para quê	Vida
`livraria:cache:*`	Vitrine, categorias, catálogos e detalhe dos livros	5 a 30 min; limpo ao mudar estoque, preço ou vitrine
`livraria:auth:refresh:<hash>`	Token de renovação de sessão	7 dias
`livraria:auth:fail:<email>`	Contador de senhas erradas	15 min
`livraria:pay:card:tok_*`	Token do cartão (bandeira e 4 últimos dígitos)	10 min, uso único
`livraria:lock:*`	Travas da expiração de pedidos e da carga inicial	até 5 min

Se o Redis for apagado, nada se perde: o cache se refaz, as pessoas entram de novo e os pedidos continuam no banco.

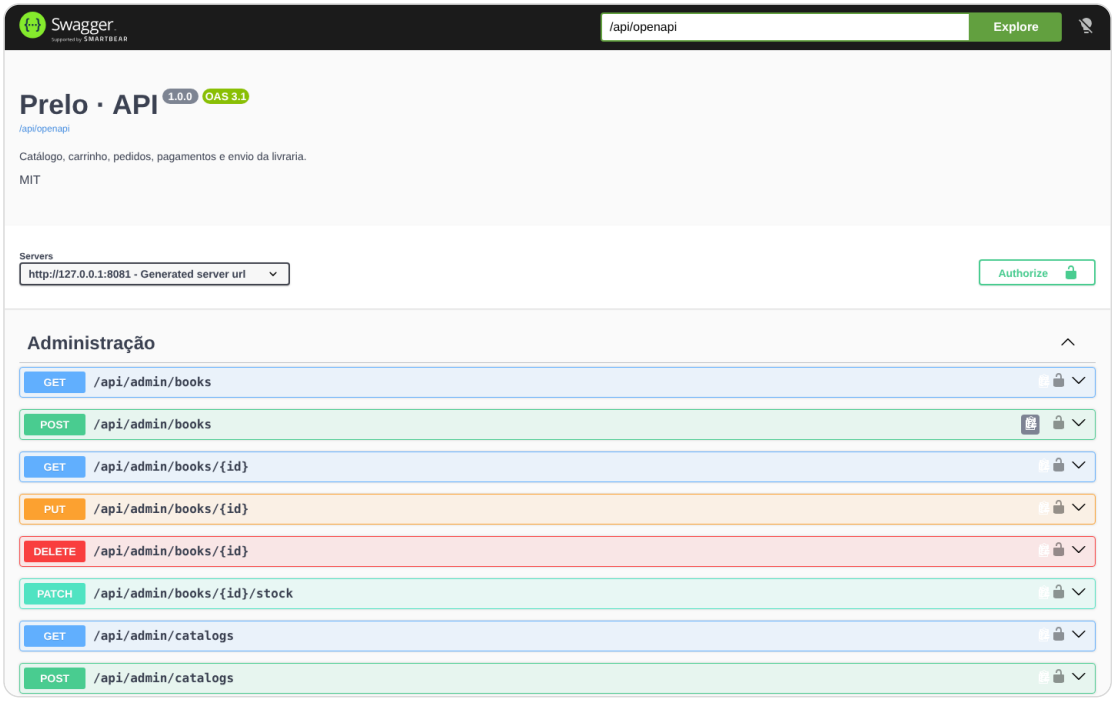
Estrutura de pastas

```
backend/
  src/main/java/io/github/violetpixel/bookstore/
    auth/ account/ catalog/ cart/ order/ payment/ shipping/ admin/ common/ config/ seed/
  src/main/resources/
    application.yml                 configuração geral
    application-<banco>.yml         URL JDBC de cada banco
    db/migration/<banco>/V1__estrutura_inicial.sql
  src/test/java/...                 testes de integração
frontend/
  src/pages/        vitrine, catálogo, livro, carrinho, checkout, compras, pedido, conta, entrada,
admin/
  src/components/   capas, cartão de livro, gaveta, janela de pagamento, linha do tempo,
formulários
  src/lib/          cliente da API, sessão, carrinho, formatação
  nginx.conf        entrega da interface e repasse de /api
docker-compose.yml  bancos por perfil, Redis, API e interface
iniciar.cmd / iniciar.sh / scripts/criar-env.ps1
```

CAPÍTULO 7

API REST

A documentação interativa fica em /api/docs. Aqui estão as rotas que a interface usa.



Swagger com todas as rotas agrupadas

Autenticação

`POST /api/auth/login` devolve `accessToken`, `expiresIn` e os dados da conta, e grava o cookie de renovação. Envie o token em `Authorization: Bearer <token>`. Quando ele expirar (401), chame `POST /api/auth/refresh` com o cookie.

Rotas principais

Rota	O que faz
`GET /api/home`	Catálogos, categorias e destaques da vitrine
`GET /api/books`	Busca: `q`, `category`, `catalog`, `format`, `minPrice`, `maxPrice`, `inStock`, `sort`, `page`, `size`
`GET /api/books/{slug}`	Detalhe com autores, ficha técnica e relacionados
`GET /api/categories` · `/api/catalogs/{slug}`	Categorias com contagem e catálogos curados
`POST /api/shipping/quote`	Frete por CEP, peso e subtotal (pública)
`POST /api/auth/register` · `login` · `refresh` · `logout` · `GET me`	Conta e sessão

Rota	O que faz
`GET/POST /api/cart` · `/items` · `/merge`	Carrinho: ver, adicionar, mudar quantidade, remover, juntar
`POST /api/checkout/quote`	Simula subtotal, cupom e opções de frete para um endereço
`POST /api/orders`	Fecha o pedido e reserva o estoque
`GET /api/orders` · `/api/orders/{codigo}`	Minhas compras e detalhe com pagamentos e envio
`POST /api/orders/{codigo}/cancel`	Cancela e estorna (antes do envio)
`GET /api/payments/methods` · `/api/orders/{codigo}/installments`	Formas de pagamento e parcelas
`POST /api/payments/card-tokens`	Troca os dados do cartão por um token de uso único
`POST /api/orders/{codigo}/payments`	Paga com `CREDIT_CARD`, `PIX` ou `BOLETO` (aceita `Idempotency-Key`)
`GET /api/payments/{referencia}`	Status de um pagamento (consultado pela janela do Pix)
`/api/account/profile` · `/password` · `/addresses`	Dados, senha e endereços
`/api/admin/dashboard` · `/orders` · `/books` · `/categories` · `/catalogs`	Administração
`POST /api/admin/orders/{codigo}/shipment/advance`	Avança o envio (com rastreio e observação opcionais)

Exemplos

```
# entrar e guardar o token
TOKEN=$(curl -s -H "Content-Type: application/json" \
  -d '{"email":"leitora@prelo.dev","password":"Prelo@2026"}' \
  http://localhost:8080/api/auth/login | jq -r .accessToken)

# livros de tecnologia, do mais barato ao mais caro
curl "http://localhost:8080/api/books?category=tecnologia&sort=price_asc&size=6"

# fechar um pedido e pagar com cartão de teste
curl -H "Authorization: Bearer $TOKEN" -H "Content-Type: application/json" \
  -d '{"addressId":1,"shippingMethod":"ECONOMICA"}' http://localhost:8080/api/orders
CARD=$(curl -s -H "Authorization: Bearer $TOKEN" -H "Content-Type: application/json" \
  -d '{"number":"4111111111111111","holderName":"TESTE","expMonth":12,"expYear":2031,"cvv":"123"}'
\
  http://localhost:8080/api/payments/card-tokens | jq -r .token)
curl -H "Authorization: Bearer $TOKEN" -H "Content-Type: application/json" -H "Idempotency-Key: $
(uuidgen)" \
  -d "{\"method\":\"CREDIT_CARD\",\"cardToken\":\"$CARD\",\"installments\":2}" \
  http://localhost:8080/api/orders/PED-261021-ABCDE/payments
```

Erros

Todo erro segue o formato Problem Details (RFC 9457) com um campo `code` estável e, em erros de validação, `fields` com a mensagem de cada campo:

```
{"status":409,"title":"Conflict","code":"estoque_insuficiente","detail":"Não há estoque suficiente de “Redis em Produção”. Ajuste a quantidade no carrinho."}
```

Status	Códigos mais comuns
400	`dados_invalidos`, `cep_invalido`, `cartao_invalido`, `parcelas_invalidas`
401	`nao_autenticado`, `credenciais_invalidas`, `sessao_expirada`
403	`sem_permissao`
404	`nao_encontrado`
409	`estoque_insuficiente`, `email_em_uso`, `pedido_nao_pendente`, `registro_alterado`
422	`carrinho_vazio`, `cupom_invalido`, `esgotado`, `prazo_expirado`, `nao_cancelavel`
429	`muitas_tentativas`

CAPÍTULO 8

Configuração

Tudo é configurado por variáveis de ambiente, lidas do .env pelo Docker Compose.

Aplicação

Variável	Padrão	O que faz
`APP_PORT`	`8080`	Porta da loja no navegador
`APP_SEED_DEMO`	`true`	Dados de demonstração num banco vazio
`APP_HTTPS`	`false`	Cookie de sessão só por HTTPS
`APP_CORS_ORIGINS`	`http://localhost:8080`	Origens liberadas para chamar a API
`API_DOCS_ENABLED`	`true`	Swagger em `/api/docs`
`APP_JOBS_ENABLED`	`true`	Tarefa de expiração de pedidos
`JWT_SECRET`	—	Chave do JWT, 32 caracteres ou mais (ou Base64 de 32 bytes)
`JWT_ACCESS_TTL` · `JWT_REFRESH_TTL`	`15m` · `7d`	Vida do token de acesso e da sessão

Banco e Redis

Variável	Padrão	O que faz
`DB_VENDOR`	`postgresql`	`postgresql`, `mysql`, `sqlserver` ou `oracle`
`COMPOSE_PROFILES`	`postgresql`	Igual a `DB_VENDOR`: qual contêiner de banco sobe
`DB_HOST` · `DB_PORT`	`db` · porta do banco	Endereço do banco (no Docker, sempre `db`)
`DB_NAME` · `DB_USER` · `DB_PASSWORD`	`prelo` · `prelo` · —	Banco, usuário e senha. No Oracle o serviço é `ORACLE_SERVICE` (padrão FREEPDB1)
`DB_URL`	—	URL JDBC completa, se preferir informar direto
`DB_POOL_SIZE`	`10`	Conexões no pool (HikariCP)
`REDIS_HOST` · `REDIS_PORT` · `REDIS_PASSWORD`	`redis` · `6379` · —	Redis

Pagamentos

Variável	Padrão	O que faz
`PAYMENTS_SANDBOX`	`true`	Cartões de teste e simulação de Pix e boleto
`PAYMENTS_ORDER_HOLD`	`60m`	Prazo para pagar um pedido (estoque reservado)
`PAYMENTS_PIX_TTL`	`30m`	Validade do QR Code Pix

Checklist para produção

1. `APP_SEED_DEMO=false` num banco vazio e a sua conta promovida a administração.
2. `JWT_SECRET` e `DB_PASSWORD` fortes e só no servidor (o `.env` nunca vai para o Git).
3. Proxy com HTTPS na frente da porta 8080 e `APP_HTTPS=true`.
4. `PAYMENTS_SANDBOX=false` com um `PaymentGateway` real.
5. `API_DOCS_ENABLED=false`, se a documentação não for pública.
6. Backup do banco (o PDF de banco de dados mostra o comando de cada um) e do volume do Redis, se quiser manter as sessões.

CAPÍTULO 9

Testes e integração contínua

Os testes da API sobem a aplicação inteira contra um banco e um Redis de verdade, nos quatro bancos.

Testes da API

Classe	O que cobre
`AuthTest`	Cadastro, e-mail repetido, login, renovação com rotação do token, saída, bloqueio após 5 senhas erradas, validação de campos, acesso negado à administração
`CatalogTest`	Vitrine (e cache), busca sem acentos, por autor e por ISBN, filtros, ordenação, paginação, detalhe, catálogo, categoria, frete por região e frete grátis
`CheckoutFlowTest`	Carrinho, cupom, checkout, baixa de estoque, cartão aprovado e recusado, token de uso único, Pix com idempotência, sandbox, Pix e boleto juntos, boleto estendendo o prazo, cancelamento com devolução do estoque, expiração, venda acima do estoque, envio até a entrega e pedidos de outra conta
`SchemaValidationTest`	Hibernate confere cada entidade contra as tabelas criadas pelo Flyway (PostgreSQL e MySQL)

```
cd backend
DB_PASSWORD=prelo mvn verify                                    # PostgreSQL local
DB_VENDOR=mysql DB_HOST=127.0.0.1 DB_PASSWORD=prelo mvn verify   # MySQL
DB_VENDOR=sqlserver DB_PASSWORD=... mvn verify                  # SQL Server
DB_VENDOR=oracle DB_PASSWORD=... mvn verify                     # Oracle (FREEPDB1)
```

Os testes usam o banco 15 do Redis, separado do da aplicação, criam contas novas a cada execução e ajustam o estoque antes de comprar, então podem rodar várias vezes no mesmo banco.

Testes da interface

```
cd frontend
npm test        # Vitest: formatação e capas geradas
npm run build   # build de produção
```

Integração contínua

O GitHub Actions roda a cada push e pull request:

Job	O que faz
API · 4 bancos	Matriz com PostgreSQL 16, MySQL 8.4, SQL Server 2022 e Oracle 23ai Free: sobe o banco, cria o usuário e roda `mvn verify`
Interface	`npm ci`, testes e build com Node 22

Job	O que faz
Docker	Valida o Compose nos quatro perfis e constrói as imagens da API e da interface

CAPÍTULO 10

Solução de problemas

O que acontece	Causa provável	O que fazer
O script diz que o Prelo ainda está subindo	Oracle e SQL Server demoram na primeira vez	Acompanhe com `docker compose logs -f api` e espere; rode o script de novo depois
`Defina DB_PASSWORD no .env`	O `.env` não tem senha	Apague o `.env` e rode o script de novo, ou preencha à mão
SQL Server não sobe	Senha fraca para o usuário sa	Use letras maiúsculas, minúsculas e números (os scripts já geram assim)
Mudei `DB_VENDOR` e subiu o banco antigo	`COMPOSE_PROFILES` diferente	Deixe os dois iguais ou rode o script, que corrige
Porta 8080 ocupada	Outro programa usa a porta	Mude `APP_PORT` no `.env`
Todo mundo cai da sessão depois de reiniciar	`JWT_SECRET` vazio fora do Docker	Defina uma chave fixa; sem ela a API usa uma temporária
CEP não preenche sozinho	Sem acesso ao serviço de CEP	Preencha à mão; o restante funciona normalmente
Pagamento com cartão dá dados expiraram	Token do cartão usado ou com mais de 10 min	Digite o cartão de novo
`estoque_insuficiente` no checkout	Alguém comprou o último exemplar antes	Ajuste a quantidade no carrinho
Pedido cancelado sozinho	Passou o prazo de pagamento	Faça o pedido de novo; o estoque já voltou
Não vejo o menu Administração	A conta é de cliente	Promova a conta no banco e entre de novo
Quero começar do zero	—	`docker compose down -v` apaga banco e Redis